

# Semiconductor Design Infrastructure Handbook

Deployment, Configuration, and Administration  
of Cadence-based RHEL workstations.

## AUTHOR

Anirban Das  
BTech (ECE)  
Class of 2027

## ADVISORS

Dr. Anshika Srivastava  
Program Coordinator  
**Department of Electronics and Communication  
Engineering**

Dr. Prasiddh Trivedi  
Assistant Professor  
**Department of Electrical Engineering**

---

## Gati Shakti Vishwavidyalaya

A Central University under Ministry of Railways, Government of India  
Vadodara, Gujarat 390004





# Abstract

*this requires further enhancement*

This manual covers the procedures required to deploy, provision, operate, maintain, and recover a RHEL-based semiconductor CAD workstation environment. It is intended to serve as an operational reference for administrators responsible for preparing workstations, installing EDA tools, configuring user access, and maintaining a reliable design environment over time.

Specifically, this manual covers the following areas:

- Operating system deployment for RHEL workstations, including installation planning and baseline system configuration.
- Workstation provisioning, including package installation, network readiness, storage preparation, and post-installation setup.
- EDA installation workflows for supported semiconductor design tools and their required dependencies.
- Licensing configuration, including license server connectivity, environment variables, and validation steps.
- Shell environments for CAD users, including startup files, module or tool setup scripts, and reproducible environment initialization.
- User management tasks, including account creation, group membership, access control, and basic permission practices.
- Troubleshooting procedures for common operating system, EDA tool, license, shell, and user-access issues.
- Recovery procedures for failed installations, broken environments, workstation rebuilds, and service restoration.
- Maintenance workflows, including updates, backups, configuration audits, cleanup tasks, and periodic validation of the CAD environment.

This manual does not cover semiconductor design methodology, RTL design, verification strategy, physical design theory, circuit design, device modeling, or EDA tool usage from a design-engineering perspective. It also does not define institutional IT policy, network architecture outside the workstation environment, procurement procedures, vendor contract negotiation, or license entitlement management beyond the technical configuration needed for workstation operation.

## Preface

Modern civilization is built upon silicon. From mobile communication systems and medical instrumentation to aerospace control systems, artificial intelligence accelerators, and high-performance computing platforms, nearly every contemporary technological system is ultimately dependent upon integrated circuits containing billions of transistors operating at nanometer-scale geometries. The design and verification of such devices would be computationally impossible without the development of sophisticated Electronic Design Automation (EDA) frameworks capable of transforming abstract circuit descriptions into physically manufacturable semiconductor layouts. Over the past several decades, EDA has evolved from a collection of isolated drafting and simulation utilities into a deeply interconnected computational ecosystem comprising schematic capture systems, hardware description language toolchains, simulation engines, parasitic extraction frameworks, physical verification suites, and advanced layout environments operating across distributed compute infrastructure. As semiconductor complexity increased, the underlying computing environments required to support these workflows evolved alongside them, giving rise to highly specialized workstation architectures optimized for reliability, reproducibility, performance, and long-term maintainability.

Contemporary semiconductor design environments are overwhelmingly deployed on Linux-based operating systems due to their stability, scalability, scripting flexibility, network integration capabilities, and compatibility with industry-standard EDA toolchains. Unlike general-purpose desktop operating systems, enterprise Linux distributions provide deterministic package management, robust permission models, multi-user infrastructure support, and predictable system behavior required for large-scale engineering workflows. Among these distributions, Red Hat Enterprise Linux (RHEL) and its derivatives have emerged as the de facto industry standard within both commercial semiconductor organizations and academic VLSI laboratories owing to their long-term support lifecycle, binary compatibility guarantees, enterprise-grade tooling ecosystem, and extensive vendor certification support from major EDA providers. Consequently, the deployment methodology described throughout this manual is centered around a standardized RHEL-based semiconductor CAD environment designed to support scalable, reproducible, and maintainable VLSI design workflows across multiple engineering workstations.

# Contents

---

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| Abstract . . . . .                                                                | i         |
| Preface . . . . .                                                                 | ii        |
| <b>I SYSTEM SETUP</b>                                                             | <b>1</b>  |
| <b>1. Foundations of the EDA Environment . . . . .</b>                            | <b>3</b>  |
| 1.1 Introduction to EDA . . . . .                                                 | 3         |
| 1.2 Why Enterprise Linux Anchors Modern EDA? . . . . .                            | 3         |
| 1.3 Red Hat Enterprise Linux as Industrial Infrastructure . . . . .               | 4         |
| 1.4 Design Philosophy of the EDA Environment . . . . .                            | 4         |
| 1.5 Overview of Laboratory Infrastructure . . . . .                               | 5         |
| 1.6 Workstation Hardware Requirements and Deployment Constraints . . . . .        | 5         |
| 1.6.1 Storage Requirements . . . . .                                              | 5         |
| 1.6.2 Memory Requirements . . . . .                                               | 6         |
| 1.6.3 Deployment Methodology: Native Installation versus Virtualization . . . . . | 6         |
| 1.7 Summary . . . . .                                                             | 7         |
| <b>2. RHEL Installation and Setup . . . . .</b>                                   | <b>9</b>  |
| 2.1 Installation Planning and Deployment Strategy . . . . .                       | 9         |
| 2.2 Hardware Requirements and Workstation Specifications . . . . .                | 10        |
| 2.3 Obtaining the RHEL Installation Media . . . . .                               | 11        |
| 2.4 BIOS, UEFI and Secure Boot Configuration . . . . .                            | 12        |
| 2.5 Virtualized RHEL Installation . . . . .                                       | 12        |
| 2.5.1 Preparing the Virtual Machine . . . . .                                     | 13        |
| 2.6 Native Dual Booting RHEL 7.9.0 . . . . .                                      | 17        |
| 2.6.1 Disk Partitioning Strategy . . . . .                                        | 17        |
| 2.6.2 Creating the Installation Media . . . . .                                   | 19        |
| 2.6.3 Installation Destination Stage . . . . .                                    | 20        |
| 2.7 Chapter Summary . . . . .                                                     | 21        |
| <b>3. Post Install Configurations . . . . .</b>                                   | <b>23</b> |
| 3.1 Username/Hostname Configuration . . . . .                                     | 23        |
| <b>4. Installing and Configuring Cadence . . . . .</b>                            | <b>25</b> |
| 4.1 Cadence Directory Structure . . . . .                                         | 25        |
| 4.2 Environment Variables . . . . .                                               | 25        |
| 4.3 C-Shell Configuration . . . . .                                               | 25        |
| 4.4 License Configuration . . . . .                                               | 25        |

|            |                                                    |           |
|------------|----------------------------------------------------|-----------|
| 4.5        | Verifying Installation . . . . .                   | 25        |
| 4.6        | Launching Cadence Tools . . . . .                  | 25        |
| <b>II</b>  | <b>RTL DESIGN FUNDAMENTALS</b>                     | <b>27</b> |
| <b>5.</b>  | <b>Introduction to Verilog . . . . .</b>           | <b>29</b> |
| 5.1        | Why Hardware Description Languages Exist . . . . . | 29        |
| 5.2        | Verilog Program Structure . . . . .                | 29        |
| 5.3        | Modules . . . . .                                  | 29        |
| 5.4        | Ports . . . . .                                    | 29        |
| 5.5        | Nets and Registers . . . . .                       | 29        |
| 5.6        | Numbers and Constants . . . . .                    | 29        |
| 5.7        | Operators . . . . .                                | 29        |
| 5.8        | Laboratory Exercise . . . . .                      | 29        |
| <b>6.</b>  | <b>Combinational Logic Design. . . . .</b>         | <b>31</b> |
| 6.1        | Continuous Assignments . . . . .                   | 31        |
| 6.2        | Dataflow Modeling . . . . .                        | 31        |
| 6.3        | Conditional Operators . . . . .                    | 31        |
| 6.4        | Multiplexers . . . . .                             | 31        |
| 6.5        | Full Adder Design . . . . .                        | 31        |
| 6.6        | Laboratory Exercise . . . . .                      | 31        |
| <b>7.</b>  | <b>Sequential Logic Design. . . . .</b>            | <b>33</b> |
| 7.1        | Why Memory is Required . . . . .                   | 33        |
| 7.2        | Clock Signals . . . . .                            | 33        |
| 7.3        | D Flip-Flops . . . . .                             | 33        |
| 7.4        | Registers . . . . .                                | 33        |
| 7.5        | Counters . . . . .                                 | 33        |
| 7.6        | Laboratory Exercise . . . . .                      | 33        |
| <b>III</b> | <b>VERIFICATION</b>                                | <b>35</b> |
| <b>8.</b>  | <b>Verilog Testbenches . . . . .</b>               | <b>37</b> |
| 8.1        | What is Verification? . . . . .                    | 37        |
| 8.2        | Testbench Architecture . . . . .                   | 37        |
| 8.3        | Initial Blocks . . . . .                           | 37        |
| 8.4        | Always Blocks . . . . .                            | 37        |
| 8.5        | Clock Generation . . . . .                         | 37        |
| 8.6        | Stimulus Generation . . . . .                      | 37        |
| 8.7        | Self-Checking Testbenches . . . . .                | 37        |
| 8.8        | Laboratory Exercise . . . . .                      | 37        |
| <b>9.</b>  | <b>Simulation and Waveform Analysis . . . . .</b>  | <b>39</b> |
| 9.1        | Compilation . . . . .                              | 39        |
| 9.2        | Elaboration . . . . .                              | 39        |
| 9.3        | Simulation . . . . .                               | 39        |
| 9.4        | Understanding Waveforms . . . . .                  | 39        |
| 9.5        | Common RTL Bugs . . . . .                          | 39        |

|                                                   |           |
|---------------------------------------------------|-----------|
| 9.6 Debugging Methodology . . . . .               | 39        |
| 9.7 Laboratory Exercise . . . . .                 | 39        |
| <b>10. Coverage Analysis using IMC . . . . .</b>  | <b>41</b> |
| 10.1 Why Simulation Alone is Not Enough . . . . . | 41        |
| 10.2 Code Coverage . . . . .                      | 41        |
| 10.3 Branch Coverage . . . . .                    | 41        |
| 10.4 Toggle Coverage . . . . .                    | 41        |
| 10.5 Functional Coverage . . . . .                | 41        |
| 10.6 Using IMC . . . . .                          | 41        |
| 10.7 Coverage Reports . . . . .                   | 41        |
| 10.8 Coverage Closure . . . . .                   | 41        |
| 10.9 Laboratory Exercise . . . . .                | 41        |
| <b>IV LOGIC SYNTHESIS . . . . .</b>               | <b>43</b> |
| <b>11. Introduction to Synthesis . . . . .</b>    | <b>45</b> |
| 11.1 RTL vs Hardware . . . . .                    | 45        |
| 11.2 Standard Cell Libraries . . . . .            | 45        |
| 11.3 Liberty Files . . . . .                      | 45        |
| 11.4 Timing Constraints . . . . .                 | 45        |
| 11.5 The Synthesis Flow . . . . .                 | 45        |
| 11.6 Reading Reports . . . . .                    | 45        |
| <b>12. Logic Synthesis using Genus . . . . .</b>  | <b>47</b> |
| 12.1 Preparing RTL . . . . .                      | 47        |
| 12.2 Writing Constraints . . . . .                | 47        |
| 12.3 Running Genus . . . . .                      | 47        |
| 12.4 Generated Outputs . . . . .                  | 47        |
| 12.5 Laboratory Exercise . . . . .                | 47        |
| <b>13. Timing Analysis . . . . .</b>              | <b>49</b> |
| 13.1 Propagation Delay . . . . .                  | 49        |
| 13.2 Setup Time . . . . .                         | 49        |
| 13.3 Hold Time . . . . .                          | 49        |
| 13.4 Clock Frequency . . . . .                    | 49        |
| 13.5 Critical Paths . . . . .                     | 49        |
| 13.6 Slack . . . . .                              | 49        |
| 13.7 Laboratory Exercise . . . . .                | 49        |
| <b>14. Logical Equivalence Checking. . . . .</b>  | <b>51</b> |
| 14.1 Why Equivalence Checking Matters . . . . .   | 51        |
| 14.2 Golden vs Revised Designs . . . . .          | 51        |
| 14.3 Conformal Workflow . . . . .                 | 51        |
| 14.4 Laboratory Exercise . . . . .                | 51        |

|            |                                        |           |
|------------|----------------------------------------|-----------|
| <b>V</b>   | <b>PHYSICAL DESIGN</b>                 | <b>53</b> |
| <b>15.</b> | <b>Introduction to Physical Design</b> | <b>55</b> |
| 15.1       | What Happens After Synthesis?          | 55        |
| 15.2       | Netlists and Standard Cells            | 55        |
| 15.3       | Die, Core and IO Regions               | 55        |
| 15.4       | Physical Design Flow Overview          | 55        |
| <b>16.</b> | <b>Floorplanning</b>                   | <b>57</b> |
| 16.1       | Defining the Core Area                 | 57        |
| 16.2       | IO Placement                           | 57        |
| 16.3       | Macro Placement                        | 57        |
| 16.4       | Utilization                            | 57        |
| 16.5       | Laboratory Exercise                    | 57        |
| <b>17.</b> | <b>Power Planning</b>                  | <b>59</b> |
| 17.1       | Why Chips Need Power Networks          | 59        |
| 17.2       | Power Rings                            | 59        |
| 17.3       | Power Stripes                          | 59        |
| 17.4       | IR Drop Concepts                       | 59        |
| 17.5       | Ground Networks                        | 59        |
| 17.6       | Laboratory Exercise                    | 59        |
| <b>18.</b> | <b>Placement</b>                       | <b>61</b> |
| 18.1       | Standard Cell Placement                | 61        |
| 18.2       | Congestion                             | 61        |
| 18.3       | Density Optimization                   | 61        |
| 18.4       | Laboratory Exercise                    | 61        |
| <b>19.</b> | <b>Clock Tree Synthesis</b>            | <b>63</b> |
| 19.1       | Clock Distribution                     | 63        |
| 19.2       | Clock Buffers                          | 63        |
| 19.3       | Clock Skew                             | 63        |
| 19.4       | Clock Latency                          | 63        |
| 19.5       | Laboratory Exercise                    | 63        |
| <b>20.</b> | <b>Routing</b>                         | <b>65</b> |
| 20.1       | Global Routing                         | 65        |
| 20.2       | Detailed Routing                       | 65        |
| 20.3       | Routing Resources                      | 65        |
| 20.4       | Signal Integrity                       | 65        |
| 20.5       | Laboratory Exercise                    | 65        |
| <b>21.</b> | <b>Physical Verification</b>           | <b>67</b> |
| 21.1       | Design Rule Checking (DRC)             | 67        |
| 21.2       | Layout Versus Schematic (LVS)          | 67        |
| 21.3       | Common Physical Violations             | 67        |
| 21.4       | Laboratory Exercise                    | 67        |



**VI TAPEOUT AND BEYOND****69**

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>22. GDSII Generation</b>                       | <b>71</b> |
| 22.1 What is GDSII?                               | 71        |
| 22.2 Manufacturing Data                           | 71        |
| 22.3 Tapeout Process                              | 71        |
| <b>23. From Silicon to PCB</b>                    | <b>73</b> |
| 23.1 Packaging                                    | 73        |
| 23.2 Pin Assignment                               | 73        |
| 23.3 Chip Integration                             | 73        |
| 23.4 PCB Design Workflow                          | 73        |
| 23.5 Why PCB Design is Different from ASIC Design | 73        |
| <b>24. Complete Design Flow Review</b>            | <b>75</b> |
| 24.1 Revisiting the Counter                       | 75        |
| 24.2 RTL → Simulation → Coverage                  | 75        |
| 24.3 Synthesis → Timing → Verification            | 75        |
| 24.4 Floorplanning → CTS → Routing                | 75        |
| 24.5 Tapeout                                      | 75        |
| 24.6 Final Flow Summary                           | 75        |



# **Part I**

# **SYSTEM SETUP**

---

Operating System Deployment and RHEL Workstation Initialization



# 01

## Foundations of the EDA Environment

---

### 1.1 Introduction to EDA

Electronic Design Automation (EDA) refers to the collection of software tools, computational frameworks, and engineering workflows used in the design, simulation, verification, and physical implementation of integrated circuits and semiconductor systems. Modern semiconductor devices contain millions to billions of transistors operating at extremely small geometries, making manual design methodologies computationally impractical and operationally impossible. EDA tools therefore serve as the foundational infrastructure enabling engineers to transform conceptual circuit designs into manufacturable silicon devices.

Contemporary EDA environments encompass a wide range of specialized applications including schematic capture systems, Hardware Description Language (HDL) development environments, circuit simulators, synthesis tools, physical layout editors, parasitic extraction frameworks, timing analysis engines, and design-rule verification systems. These tools collectively support the complete semiconductor development lifecycle, from behavioral modeling and functional verification to physical layout generation and fabrication validation.

The increasing complexity of Very Large Scale Integration (VLSI) systems has made EDA indispensable within both academic and industrial semiconductor design workflows. Beyond improving productivity, EDA enables precision, scalability, repeatability, and verification accuracy that would otherwise be unattainable through conventional design methodologies. As a result, modern semiconductor engineering is fundamentally dependent upon robust EDA infrastructure and the computational environments that support it.

### 1.2 Why Enterprise Linux Anchors Modern EDA?

Modern EDA toolchains impose demanding requirements on the underlying operating system environment, including stability, deterministic behavior, scripting flexibility, network interoperability, multi-user access control, and long-term software compatibility. Consequently, semiconductor design organizations and research laboratories overwhelmingly deploy EDA infrastructure on Linux-based operating systems rather than general-purpose desktop platforms.

Linux provides a highly modular and administratively transparent environment capable of supporting complex engineering workflows involving distributed compute resources, centralized licensing systems, shared storage infrastructure, remote administration, and automated deployment methodologies. Native shell scripting capabilities, package management systems, process control utilities, and network tooling further enable scalable workstation administration and reproducible environment configuration across large laboratory deployments.

Among available Linux distributions, enterprise-grade platforms such as **Red Hat Enterprise Linux (RHEL)** and its derivatives have become the de facto standard within semiconductor design environments. Their widespread adoption is driven by long-term support lifecycles, predictable release stability, extensive vendor certification, and broad compatibility with commercial EDA applications. Most major semiconductor

tool vendors officially validate and support their software primarily on enterprise Linux environments, making RHEL-based systems the preferred foundation for reliable and maintainable CAD infrastructure deployment.

Accordingly, the workstation architecture described throughout this manual is built upon a standardized RHEL-based environment designed to provide operational consistency, scalability, and long-term maintainability for semiconductor design laboratories and VLSI research workflows.

## 1.3 Red Hat Enterprise Linux as Industrial Infrastructure

The decision to standardize a semiconductor design environment on a specific Linux distribution is not a matter of administrative preference. It is an infrastructure decision with direct consequences for EDA tool compatibility, operational stability, vendor support, and long-term maintainability. Within professional semiconductor environments, Red Hat Enterprise Linux has become a preferred foundation because it provides the stability and compatibility characteristics required by commercial EDA workflows.

Commercial EDA tools are typically distributed as precompiled binary applications that depend on stable system libraries, predictable runtime behavior, and well-defined operating system interfaces. RHEL maintains Application Binary Interface and Application Programming Interface consistency within a major release, allowing validated tool installations to remain reliable across routine system updates. This consistency is especially important for tools that depend on components such as `glibc`, `libX11`, Motif libraries, OpenSSL, shell utilities, and kernel-level behavior.

RHEL also provides long-term lifecycle stability that aligns well with semiconductor design workflows. A laboratory or production design environment may depend on a specific operating system and EDA tool combination for several years. Frequent distribution-level changes can introduce dependency drift and invalidate previously working configurations. By contrast, RHEL's enterprise release model prioritizes predictable maintenance, security updates, and controlled package evolution over rapid upstream change.

Vendor certification is another important factor. Major EDA vendors commonly certify their tools against specific RHEL major versions and closely compatible derivatives. Operating within a certified platform reduces support risk, simplifies troubleshooting, and provides a defensible baseline when vendor escalation is required. While unsupported Linux distributions may sometimes run the same tools, they introduce additional uncertainty into installation, execution, and support workflows.

The RHEL ecosystem also supports maintainable administration practices through standard tools such as `dnf`, `systemd`, `lvm`, `sssd`, and `subscription-manager`. These tools provide a well-documented administrative surface for package management, service control, storage configuration, authentication integration, and system maintenance. In institutional environments where systems must be rebuilt, audited, extended, or handed over between administrators, this consistency is a practical advantage.

Throughout the remainder of this manual, operating system procedures, configuration examples, package names, service management commands, and administrative workflows are specified for a RHEL 9-compatible environment. Unless otherwise noted, the procedures also apply to Rocky Linux 9 and AlmaLinux 9 as binary-compatible RHEL derivatives. References to *RHEL* in the operational context of this document should therefore be understood to include these derivatives unless a Red Hat subscription-specific feature is explicitly required.

## 1.4 Design Philosophy of the EDA Environment

The semiconductor design infrastructure described in this manual is not assembled opportunistically. It is engineered according to a defined set of operational principles that govern every decision from initial workstation deployment through long-term maintenance. Understanding these principles is necessary context for the procedures that follow, because many configuration choices that might otherwise appear arbitrary are direct expressions of the underlying philosophy.

The environment is built around **standardization**. All workstations in the laboratory are deployed from a common baseline: identical operating system version, identical package selection, identical directory structures, and identical tool installation paths. Deviation from this baseline is treated as an exception requiring justification, not a routine outcome of per-machine administration. Standardization is the prerequisite for everything else.

From standardization follows **reproducibility**. A workstation rebuilt from documented procedures should produce an environment functionally identical to the one it replaces. This property matters both operationally—when a workstation fails and must be restored—and institutionally, as the laboratory’s administrative knowledge must remain embedded in its documentation rather than in the accumulated state of individual machines.

The deployment is designed to **scale**. Procedures written for a single workstation apply without modification to the full laboratory fleet. Configuration that must be individually customized per machine is a design defect. Where centralized mechanisms exist—for user authentication, license server connectivity, shared storage, or environment initialization—they are used in preference to local equivalents.

**Centralized administration** follows from this. User accounts, access permissions, and shell environments are managed at the infrastructure level rather than configured locally on each workstation. This reduces the administrative surface area, eliminates inconsistency across machines, and ensures that changes propagate uniformly rather than requiring repeated per-host intervention.

Underlying all of these principles is the explicit goal of **eliminating configuration drift**. In long-running laboratory environments, workstations tend to diverge from one another over time as ad hoc fixes, local experiments, and undocumented modifications accumulate. The procedures in this manual are written to resist this tendency—through documented baselines, version-controlled configuration, and periodic validation practices that detect and correct deviation before it compounds.

The result is an infrastructure in which any workstation can be administered, diagnosed, or rebuilt by any administrator with access to this document, without dependence on institutional memory or machine-specific knowledge.

## 1.5 Overview of Laboratory Infrastructure

## 1.6 Workstation Hardware Requirements and Deployment Constraints

Electronic Design Automation tools impose system resource requirements that differ substantially from those of general-purpose desktop software. A typical office productivity environment or software development workstation can operate adequately on modest storage allocations, a few gigabytes of memory, and commodity processor performance. EDA environments do not share these characteristics. The software stacks involved in a complete semiconductor design workflow are architecturally large, computationally intensive, and operationally demanding in ways that must be accounted for at the infrastructure planning stage rather than addressed reactively after deployment.

### 1.6.1 Storage Requirements

Modern EDA suites consist of numerous interrelated tools, each with its own binaries, libraries, technology files, process design kits, documentation sets, and supporting scripts. A complete installation image for a commercial EDA suite does not occupy gigabytes in the way that a large application suite might. It occupies an order of magnitude more. The Cadence EDA installation deployed in this laboratory environment occupied approximately **480 GB** of storage prior to the creation of any user project data, simulation output, waveform databases, or temporary working files. This figure reflects only the tool installation itself.

In practice, active design work compounds storage consumption considerably. Simulation runs generate large output datasets. Layout databases accumulate across design iterations. Waveform viewers load and cache substantial binary files during interactive sessions. Verification engines write intermediate results to disk throughout their execution. Over the operational lifetime of a workstation, the storage footprint of the design environment grows continuously. Storage planning must therefore account not only for the initial

installation image but for a realistic projection of working data volume across the deployment period.

For this reason, workstations intended for sustained EDA use should be provisioned with local storage that provides meaningful headroom beyond the installation baseline. The 480 GB installation figure should be treated as a floor, not a target. Configurations that provision only enough storage to accommodate the initial installation leave no margin for project data, OS overhead, swap allocation, or tool updates, and will require administrative intervention as the environment matures.

### 1.6.2 Memory Requirements

Semiconductor design workflows routinely involve multiple concurrently active applications. A typical interactive session may involve a layout editor, a schematic capture environment, a circuit simulator, a waveform viewer, and a design rule checking utility operating simultaneously within the same desktop session. Each of these applications maintains its own working set in memory, and several—particularly simulation engines and verification utilities—can expand their memory consumption substantially during active computation.

Insufficient memory allocation produces predictable and disruptive consequences in this context. Operating system memory pressure forces active application data to swap, degrading interactive responsiveness and extending simulation runtimes. In severe cases, the out-of-memory subsystem may terminate running processes, corrupting simulation state and requiring workflow restart. These outcomes are not edge cases in underpowered environments; they are routine operational hazards.

Workstations deployed for full EDA workflows should be provisioned with sufficient physical memory to support concurrent tool operation without inducing swap utilization under normal working conditions. The specific requirement varies by workflow, but **configurations below 16 GB of physical memory are generally unsuitable for sustained multi-tool VLSI design sessions**. Workstations supporting heavier simulation workloads or concurrent multi-user access benefit from higher memory allocations accordingly.

### 1.6.3 Deployment Methodology: Native Installation versus Virtualization

The manner in which RHEL is deployed on a workstation has direct consequences for the resource availability of the EDA environment. Two deployment models are relevant to this laboratory context: direct installation on dedicated hardware, and virtualized deployment within a hypervisor environment running on an existing host operating system.

In a native installation—whether on a dedicated workstation or in a dual-boot configuration alongside an existing operating system—RHEL has unrestricted access to all physical hardware resources. The full complement of CPU cores, physical memory, storage throughput, and graphics capability is available to the operating system and the EDA tools running within it. This is the preferred deployment model for sustained engineering workflows, as it eliminates the resource-sharing constraints and performance overhead inherent to virtualized environments.

Virtualization, however, represents a practically useful alternative under specific deployment conditions. During the initial workshop phase of this laboratory deployment, Microsoft Hyper-V was used to provision RHEL virtual machines on workstations running existing Windows installations. This approach was selected because many of the available systems contained active Windows environments that could not be immediately repartitioned, and the operational priority was to make the EDA environment accessible within a constrained preparation timeline.

The resource tradeoffs introduced by this approach are significant and should be understood clearly. The reference deployment system carried 32 GB of physical memory. Of this, 16 GB was allocated to the RHEL virtual machine, with the remaining capacity reserved for the Windows host and the memory overhead of the Hyper-V hypervisor layer itself. Under this configuration, the EDA environment operates within a resource envelope that is materially smaller than the physical hardware could otherwise provide. CPU scheduling is mediated by the hypervisor, storage I/O is subject to virtualization overhead, and memory allocation is fixed at provisioning time rather than dynamically available from the full physical pool.



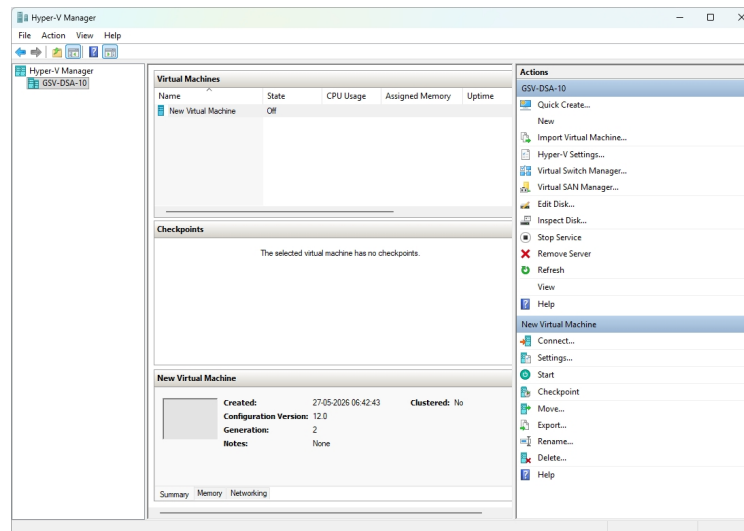


Figure 1.1: Screenshot of Hyper-V

Virtualized deployment is appropriate for introductory workshops, environment familiarization, and short-duration instructional use cases in which the full performance envelope of the EDA tools is not required. It is not the preferred model for production semiconductor design workflows. As workstation preparation schedules permit, systems initially deployed under Hyper-V should be transitioned to native RHEL installations in order to provide the EDA environment with unrestricted access to the underlying hardware resources. All procedures documented in this manual apply to both deployment models unless a specific section explicitly notes otherwise.

## 1.7 Summary

This chapter introduced the foundational concepts underlying modern semiconductor EDA infrastructure and established the operational context for the deployment procedures described throughout this manual. The discussion examined the role of Electronic Design Automation within contemporary VLSI workflows, the infrastructure requirements imposed by commercial EDA toolchains, and the reasons enterprise Linux environments have become the standard platform for semiconductor engineering systems.

The chapter further justified the adoption of Red Hat Enterprise Linux as the primary operating system environment for workstation deployment, emphasizing considerations such as long-term stability, vendor certification, maintainability, and compatibility with commercial EDA applications. In addition, the architectural philosophy of the laboratory environment, including standardization, reproducibility, centralized administration, and scalability, was introduced as the guiding framework for all subsequent deployment and maintenance procedures.

Finally, the chapter discussed the hardware and deployment considerations associated with EDA environments, including storage requirements, memory allocation strategies, and the operational tradeoffs between native and virtualized deployment methodologies. These concepts collectively establish the technical foundation required for the operating system installation and workstation provisioning procedures described in the following chapter.



# 02

## RHEL Installation and Setup

---

**Note:** This guide assumes you are familiar with Linux environments to a basic degree, are comfortable with the terminal, and have an active internet connection. We will not discuss the entire installation process, only the core infrastructure decisions that need to be made for successful setup of the base system.

That being said however, every decision and command will be discussed in elaborate detail – justifying why it was made, and what it does.

### 2.1 Installation Planning and Deployment Strategy

Before a workstation is installed, the deployment method must be selected deliberately. The choice is not merely a matter of convenience; it determines how the machine will consume hardware resources, how reliably EDA tools will interact with the operating system, how easily the workstation can be maintained, and how closely the final environment will resemble a production semiconductor design platform. In the context of this manual, RHEL is treated as the operating system baseline for a controlled EDA workstation rather than as a general-purpose desktop installation.

There are two practical ways to deploy RHEL on a laboratory workstation:

1. **Virtualization** – installing RHEL inside a virtual machine hosted by an existing operating system.
2. **Dual booting** – installing RHEL directly on the physical machine alongside an existing operating system by using bootable installation media.

Virtualization is useful for training, experimentation, documentation, and low-risk validation of administrative procedures. It allows the installer to be tested without repartitioning the physical disk and makes it easy to take snapshots before major configuration changes. However, virtual machines introduce an additional abstraction layer between the EDA workload and the hardware. This can limit graphics performance, memory availability, disk throughput, USB access, license-dongle support, and other device-level behavior that may matter during tool deployment or troubleshooting.

Dual booting installs RHEL natively on the workstation hardware. This approach gives the Linux environment direct access to the CPU, memory, graphics adapter, storage devices, and network interfaces, making it the preferred method when the machine is expected to function as a serious EDA workstation. Native installation also more closely matches the assumptions made by commercial EDA vendors, whose installation and support documentation typically targets physical or enterprise-managed Linux systems rather than desktop virtual machines.

The decision made at this stage affects the entire downstream configuration: disk layout, package selection, driver support, user account preparation, license-server connectivity, shared storage mounts, backup strategy, and future system recovery. For that reason, the installation procedure should be selected before any

partitioning or media preparation begins. This chapter documents the installation workflow with attention to university licensing constraints, C2S operational guidelines, and the need for a reproducible workstation baseline.

## 2.2 Hardware Requirements and Workstation Specifications

A modern semiconductor CAD workstation intended for full-scale Cadence deployment must be provisioned with significantly higher hardware resources than a conventional desktop system. Contemporary EDA workflows involve computationally intensive applications for schematic capture, mixed-signal simulation, physical layout, parasitic extraction, verification, and waveform analysis, many of which operate on large design databases and generate substantial intermediate data during execution. In addition to the operating system itself, a complete Cadence installation together with process design kits (PDKs), technology libraries, simulation environments, temporary build data, and user project directories can consume several hundred gigabytes of storage.

Workstations intended for academic laboratory deployment should therefore prioritize memory capacity, storage performance, multi-core CPU capability, thermal stability, and long-term maintainability over consumer-oriented graphical features. While smaller configurations may support introductory workflows, a poorly provisioned workstation can significantly degrade simulation performance, layout responsiveness, and overall user productivity during large-scale VLSI design tasks.

The recommended baseline for a Cadence-based EDA workstation is summarized below:

- **Processor:** Modern multi-core 64-bit processor with virtualization support; Intel Core i7, AMD Ryzen 7, or higher is recommended, with 8 cores preferred.
- **Memory:** Minimum 16 GB RAM for basic educational workflows; 32 GB or higher is recommended for large simulations and multi-user laboratory use.
- **Primary storage:** Minimum 500 GB SSD dedicated to Linux and EDA tools; 1 TB NVMe SSD is preferred for better database and simulation performance.
- **Secondary storage:** Optional high-capacity HDD or NAS-backed storage for project archives, backups, and shared design data.
- **Graphics:** A discrete GPU is generally not required for most Cadence workflows, but stable OpenGL support is recommended for layout visualization.
- **Display:** 1920×1080 minimum resolution; dual monitors are strongly recommended for schematic, layout, and waveform analysis.
- **Network:** Gigabit Ethernet is strongly recommended for license-server communication, shared storage access, and centralized administration.
- **Firmware:** UEFI-capable system preferred with configurable Secure Boot settings.
- **Operating system:** RHEL 7.9 or an institution-approved RHEL-compatible enterprise Linux distribution.

At the time of writing of this document, the current system available to us had the following specs:

| Component               | Detected Specification                                                                                              |
|-------------------------|---------------------------------------------------------------------------------------------------------------------|
| Workstation Model       | HP Z4 G5 Workstation Desktop PC                                                                                     |
| System Type             | 64-bit x86 Workstation Platform                                                                                     |
| Processor (CPU)         | Intel Xeon w3-2423 Processor<br>6 Physical Cores, 12 Logical Processors<br>Base Frequency: 2.11 GHz                 |
| Installed Memory (RAM)  | 32 GB DDR Memory Installed                                                                                          |
| Firmware / BIOS Mode    | UEFI Boot Mode Enabled                                                                                              |
| Secure Boot Status      | Enabled                                                                                                             |
| Virtualization Support  | Hardware Virtualization Enabled<br>Hypervisor and Virtualization-Based Security Active                              |
| Motherboard Platform    | HP System Board 8962                                                                                                |
| Operating System        | Microsoft Windows 11 Pro for Workstations<br>Build 26200 (64-bit)                                                   |
| Platform Classification | Enterprise Workstation-Class System                                                                                 |
| Storage Configuration   | Not fully listed in exported report<br>Recommended verification of SSD/NVMe capacity<br>before EDA deployment       |
| Graphics Processing     | Not listed in current report extract<br>Dedicated GPU verification recommended for<br>OpenGL-based layout workflows |

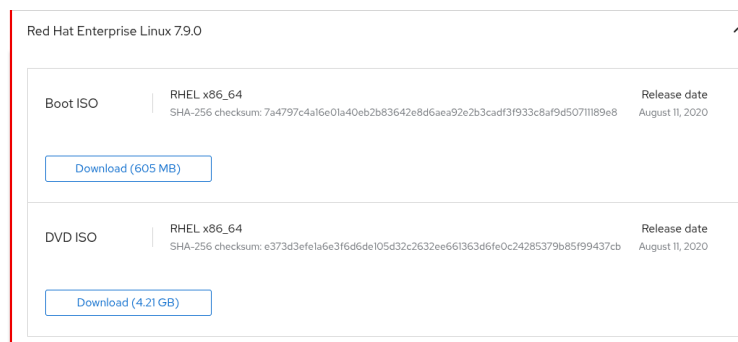
Table 2.1: Host Workstation Specifications Prior to RHEL Deployment

## 2.3 Obtaining the RHEL Installation Media

The installation workflow begins with obtaining the correct RHEL installation image from Red Hat. A Red Hat Developer account can be used for non-production developer access, subject to Red Hat's current subscription terms and the institution's licensing policy. After signing in, the installer image should be downloaded from the Red Hat Developer portal:

<https://developers.redhat.com/products/rhel/download>

For this workstation deployment, we are using **Red Hat Enterprise Linux 7.9.0**. This version is selected because many EDA tools and supporting libraries are validated against older enterprise Linux releases, and RHEL 7.9.0 provides a mature, well-documented platform with broad community and vendor support. In an EDA environment, compatibility and predictable behavior are usually more important than using the newest available operating-system release.



**Make sure to download the DVD not the Boot version.** The downloaded ISO should match the architecture of the workstation, normally x86\_64 for standard laboratory PCs. The version should also match the operating-system baseline approved for the EDA toolchain being deployed. Once the ISO has been downloaded, its integrity should be verified when checksum information is available, and the file should be stored in a documented location so that future workstation rebuilds use the same installation source.

Before writing the ISO to a USB drive or modifying the workstation disk, record the machine's current boot mode, storage configuration, and existing operating-system layout. In particular, confirm whether the system is using UEFI or legacy BIOS boot, whether Secure Boot is enabled, how much unallocated disk space is available, and whether the disk uses GPT or MBR partitioning. These details determine the correct installation path and reduce the risk of inconsistent dual-boot behavior across machines.

## 2.4 BIOS, UEFI and Secure Boot Configuration

Before installing RHEL, it is important to understand the firmware layer that controls how the machine starts. The firmware initializes hardware, detects bootable devices, and hands control to the operating-system bootloader. Incorrect firmware settings can prevent the installer from booting or can cause the installed operating system to disappear from the boot menu.

**BIOS** refers to the older firmware interface used by traditional PCs. It performs basic hardware initialization and boots from disks using the legacy boot process, commonly associated with MBR partitioning. Modern workstations generally no longer depend on legacy BIOS, but the term is still commonly used to refer to the system firmware setup screen.

**UEFI** is the modern replacement for legacy BIOS. It supports GPT partitioning, an EFI System Partition, graphical firmware menus, larger disks, and more flexible boot management. For current workstation deployments, RHEL should normally be installed in UEFI mode so that it matches the boot mode used by Windows and avoids mixed-mode dual-boot problems.

**Secure Boot** is a UEFI security feature that allows only trusted, signed bootloaders and kernel components to start during the boot process. It helps protect the system from unauthorized boot-time code, but it can also affect Linux installation, third-party drivers, and some virtualization workflows if not configured correctly.

**Note:** On the HP workstation used in this deployment, the firmware setup screen can be entered by pressing **F2** during startup. On other systems, the key may be different; common alternatives include **F10**, **F12**, **Esc**, or **Delete**, depending on the manufacturer.

Secure Boot will be discussed in more detail in the next section, separately for native dual-boot installation and virtualization-based deployment. The correct setting depends on the installation model, the bootloader path, and whether the deployment requires vendor drivers or kernel-level components that may not be signed for Secure Boot.

## 2.5 Virtualized RHEL Installation

A virtualized RHEL installation is useful for evaluating the installer, documenting setup procedures, testing package installations, and training users without modifying the physical workstation disk. In this approach, RHEL runs as a guest operating system inside a hypervisor such as VMware Workstation, VirtualBox, or Hyper-V, while Windows remains the host system, making the process low risk because the virtual disk exists only as a file that can be backed up, restored, or removed easily.

Although virtualization accurately reproduces the RHEL userspace, shell environment, package management, and administrative workflow, it does not fully replicate native hardware behavior, particularly for graphics acceleration, USB passthrough, storage performance, license dongles, and memory-intensive EDA workloads. Therefore, virtualization is best suited for validation, learning, and preliminary testing, while

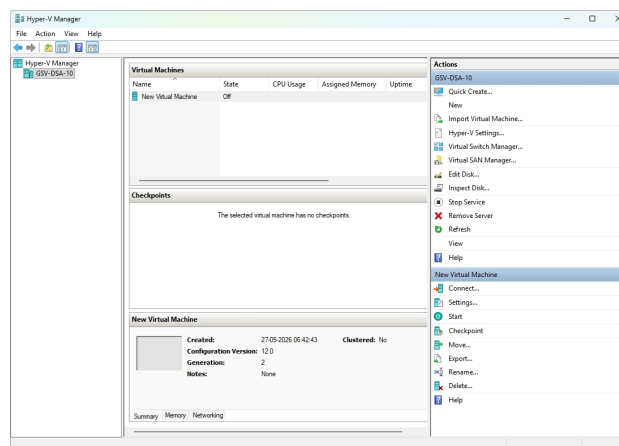
native dual booting remains the preferred choice for production-scale Cadence usage. Before creating the VM, ensure that hardware virtualization support (Intel VT-x or AMD-V/SVM) is enabled in the system firmware.

### 2.5.1 Preparing the Virtual Machine

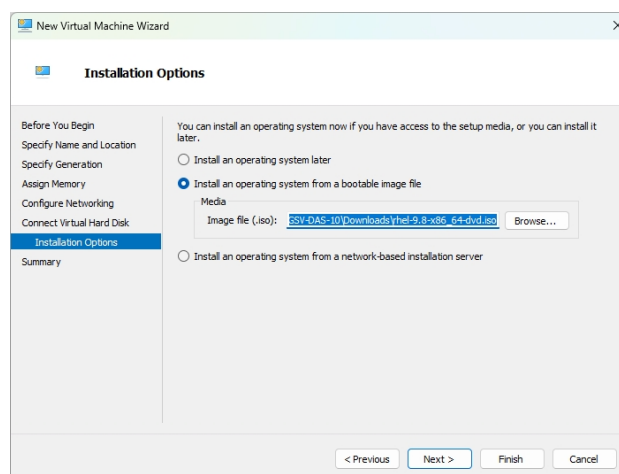
We will need to enable Microsoft Hyper-V for this. Click **Start -> Turn Windows Features On or Off**, scroll till you find the Hyper-V icon and check the box. Restart the system once.

**Note:** You need Windows Pro or Enterprise to be able to enable Hyper-V. On Windows-Home Hyper-V is not natively supported, but can be installed via a script provided at the end of this chapter.

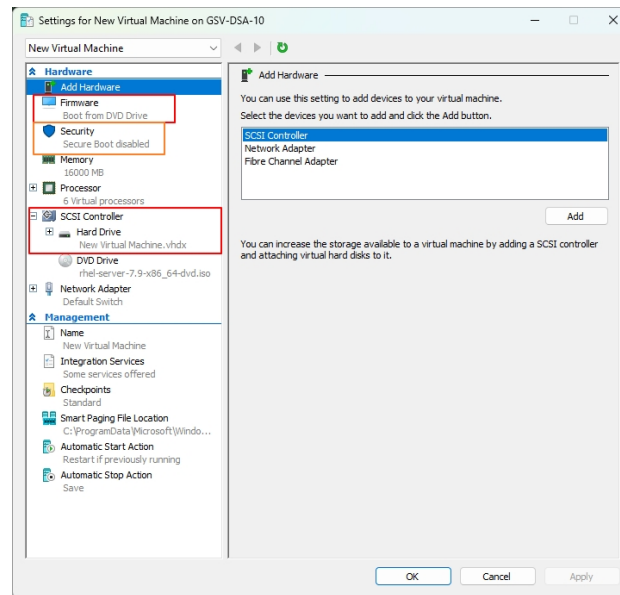
Once you have restarted the system once, open Hyper-V, you will be met with the following screen:



Click "New" → "Virtual Machine" and follow the prompts further. Make sure you choose the correct Installation Media – the ISO file we installed earlier.

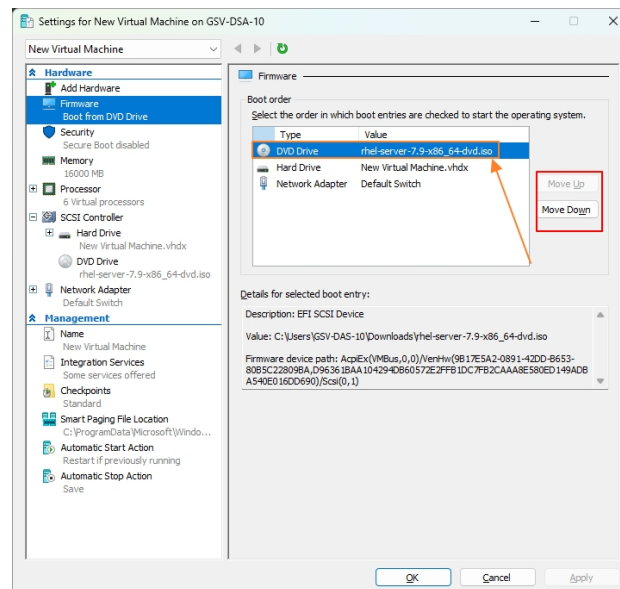


Most of the steps in the prompt are straight-forward. There are a few key configurations you need to keep in mind after setting up the virtual Machine. **Right Click** on the virtual machine, and then click **Settings** to open the following window:



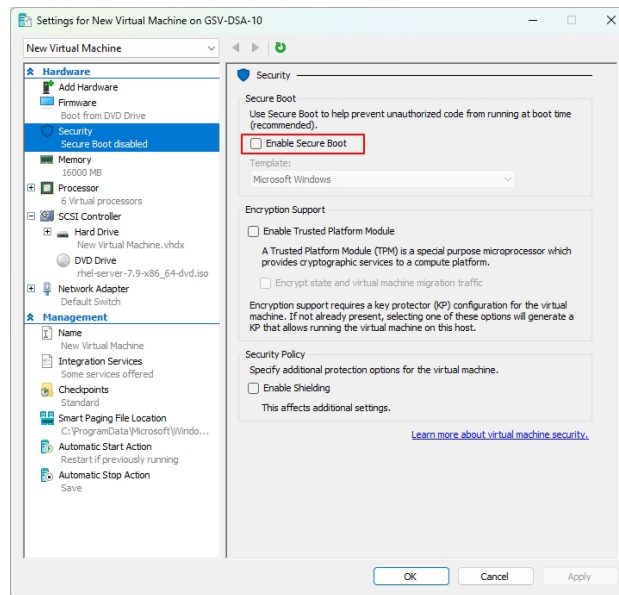
We're interested in only the following three topics, namely:

1. Firmware – make sure that DVD drive is at the top of the hierarchy.

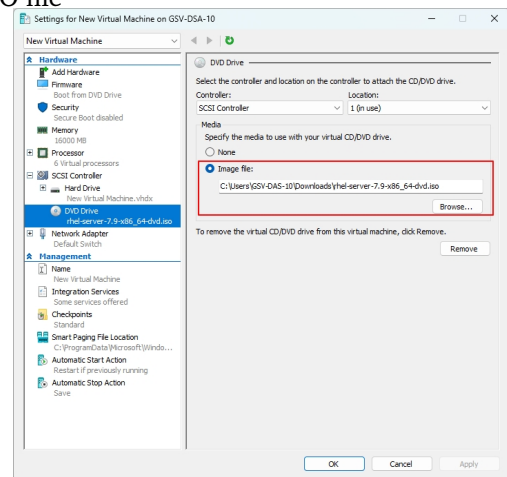
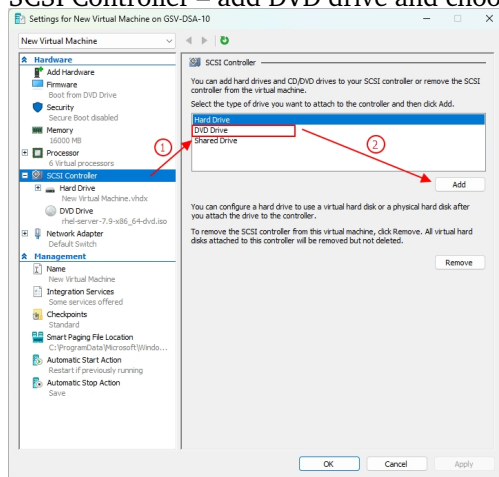


2. Security – disable Secure Boot

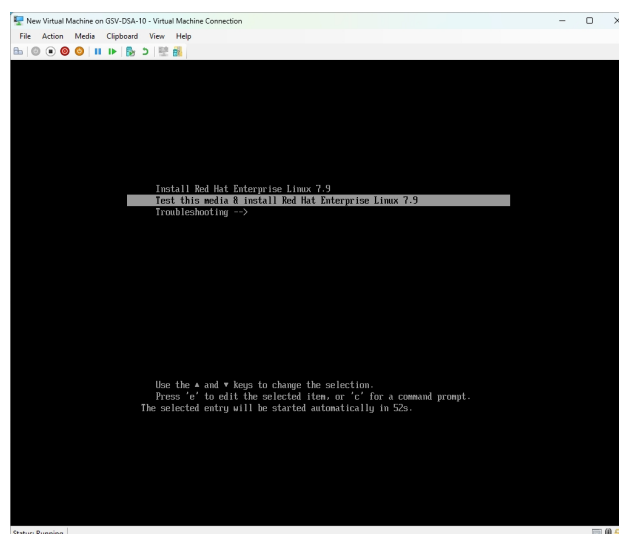




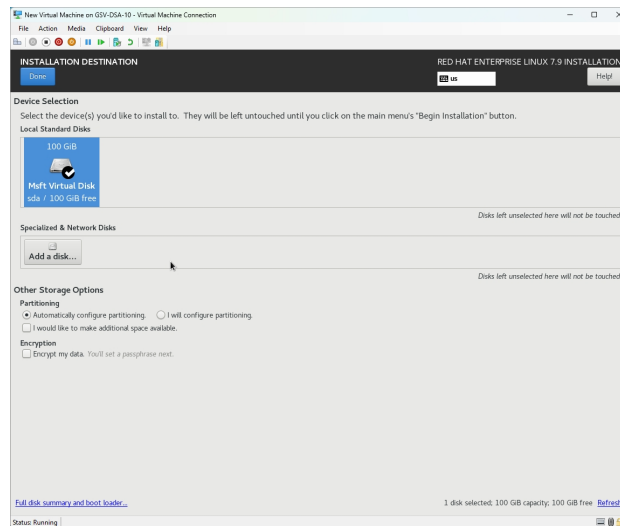
### 3. SCSI Controller – add DVD drive and choose the RHEL ISO file



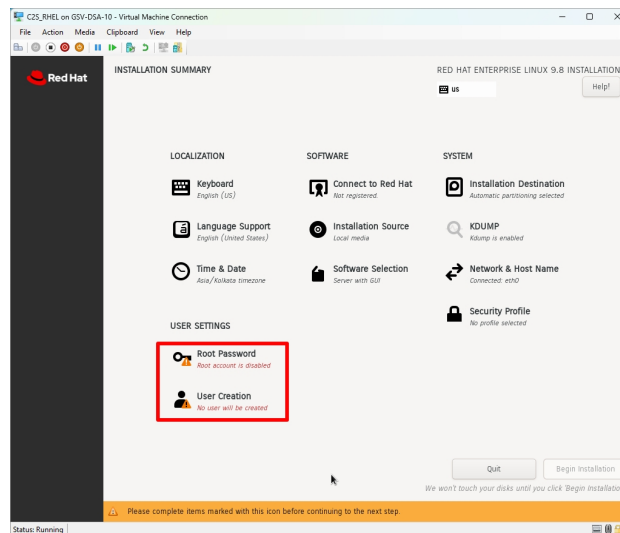
After that close the window and click **Start** and then **Connect**. You should see the following screen:



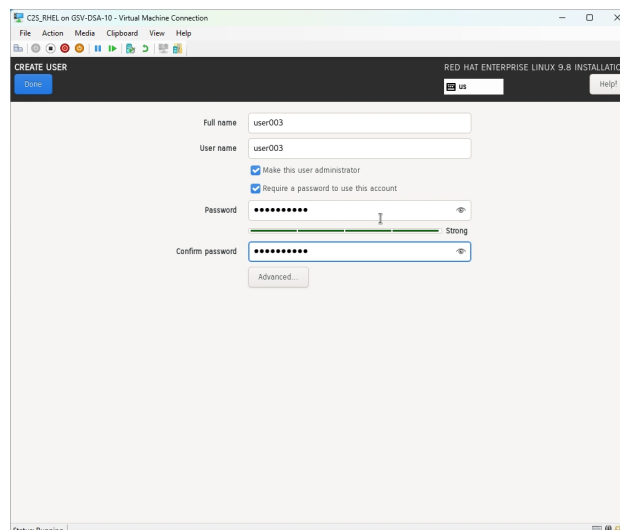
Click **"Install Red Hat Enterprise Linux 7.9"**, and follow the setup prompts. For the **"Installation Destination"** on the Virtual Machine, it should automatically select the Virtual Disk Space (.vhd) you set while setting up the Virtual Machine.



After all this, just one crucial thing is remaining – the username. Click the following cards in the red box, shown on the image on the left below:

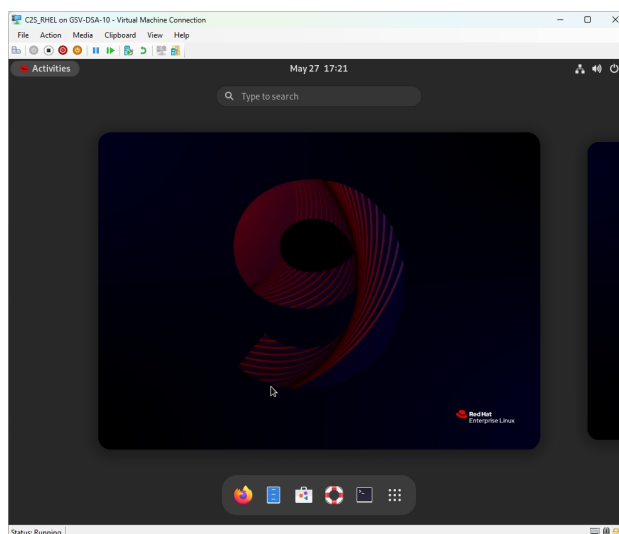


For our use case, we're using a placeholder user named **user003**. This username has been chosen for a specific reason, as will be explained in elaborate detail later in this chapter:



It is generally wise to keep different root and local user passwords, but since we're setting up a workstation for a Laboratory, we keep the passwords the same.

The installation is ready to begin. After the final installation you will be prompted to reboot the virtual Machine. Finally you should see a screen similar to this: (*this is the GNOME desktop environment*)



## 2.6 Native Dual Booting RHEL 7.9.0

Dual booting allows RHEL to run directly on the workstation while preserving an existing operating system on a separate partition. This is especially useful in academic laboratories where a machine may need to support both Linux-based EDA workflows and existing instructional or administrative software. The dual-boot model must be planned carefully, because mistakes during partitioning or bootloader configuration can make the existing operating system temporarily unbootable or, in the worst case, destroy data. Important user files should therefore be backed up before installation begins.

### 2.6.1 Disk Partitioning Strategy

This is a critical stage in the overall system deployment pipeline and must be approached with precision and caution. Any misconfiguration at this level can result in irreversible data loss, including corruption or removal of existing operating systems such as Windows.

Disk partitioning is the process of logically dividing a physical storage device (SSD or HDD) into multiple independent regions, known as partitions. Each partition behaves as a separate storage volume and is assigned a specific role within the operating system architecture.

In Linux-based system design, partitioning is not arbitrary. It is a structured layout intended to separate system-critical components, improve stability, simplify recovery, and support long-term maintainability of the environment.

A standard Linux workstation deployment typically includes the following core partitions:

1. **EFI Partition** – *mounted at /boot/efi/*  
This is a mandatory boot partition on UEFI-based systems. It stores bootloaders, firmware interfaces, and essential startup binaries required for system initialization. Without this partition, the system cannot boot in a modern UEFI environment.
2. **Root Partition** – *mounted at /*  
This is the primary filesystem hierarchy. It contains the operating system, installed packages, system binaries, configuration files, and most runtime environments. It effectively forms the core of the Linux installation.

### 3. Swap Partition

This is a dedicated memory extension area used when physical RAM is fully utilized. It supports memory paging, system hibernation (if enabled), and prevents abrupt failures under memory pressure. While modern systems often use swap files, a dedicated swap partition is preferred in controlled workstation environments for predictability and performance consistency.

You may need to manually create a partition based on your system configuration. Press **Windows Key** and then type **Disk Management**. You should see the following:

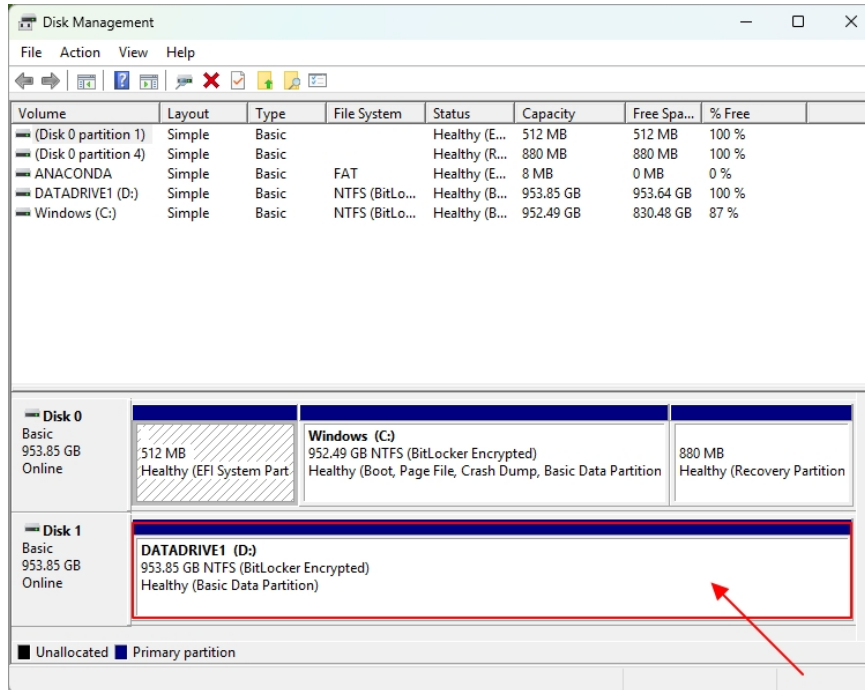


Figure 2.1: Disk Management

From the screenshot, we already have a separate disk — **Disk 1**, with a capacity of **1 TB**. In this case, we can directly use the entire disk without creating additional partitions. However, in the case of a single shared disk, it may be necessary to shrink existing unallocated space in order to create a new partition for installation or data separation.

A detailed walkthrough of this process can be found here: <https://www.youtube.com/watch?v=Wv0Id5CRmw8>.

For this demonstration, we will skip the partitioning step. The key point of interest here is **disk nomenclature in Linux**. Unlike Windows, which uses drive letters such as C: or D:, Linux represents storage devices using a structured naming convention. For NVMe SSDs, the naming format is:

`nvmeXnYpZ`

where:

- X denotes the NVMe controller number (i.e., the physical interface index),
- Y denotes the namespace number (a firmware-defined logical storage unit),
- Z denotes the partition number (a software-defined subdivision of the namespace).

Thus, from the observed system output, we can infer the following mappings:

**Disk 0**    →    **nvme0n1**  
**Disk 1**    →    **nvme1n1**

Furthermore, partitions within these disks are represented as extensions of the namespace identifier. For example:

|                     |   |           |
|---------------------|---|-----------|
| Disk 0, Partition 1 | → | nvme0n1p1 |
| Disk 0, Partition 2 | → | nvme0n1p2 |
| Disk 1, Partition 1 | → | nvme1n1p1 |

Here, the namespace (nY) represents a firmware-defined logical block device exposed by the NVMe controller, while partitions (pZ) are operating system-defined subdivisions within that namespace used for filesystem organization and data separation.

**Note:** The important thing here to remember the name of the disk we want to install RHEL on. In our case, its **nvme1n1**. The mount-point will be at **/dev/nvme1n1**

### 2.6.2 Creating the Installation Media

For dual-booting, we need to create something known as a "**bootable-drive**". The process is straight-forward. All you need are a USB with atleast 8GB storage capacity, and the ISO we downloaded earlier. To create a bootable-drive, you need a flashing tool.

For Windows, the most popular choice is Rufus; can be installed here: <https://rufus.ie/en/>. If you're on Linux, the most popular choice is Balena Etcher; can be installed here: <https://etcher.balena.io/>.

Plug in your USB and make sure it's mounted. Rufus automatically detects it.

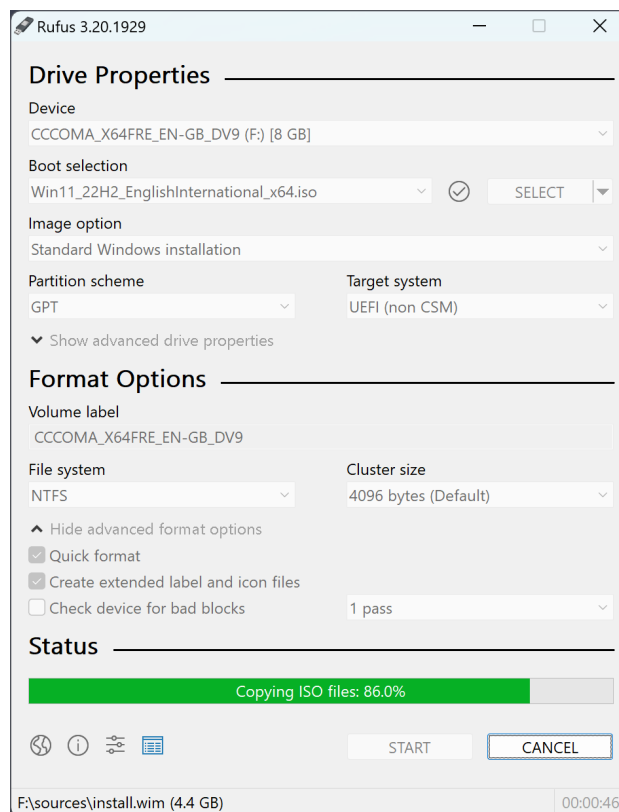


Figure 2.2: Example Screenshot of Rufus

This is a screenshot of the Rufus Tool. The *Device* is usually automatically selected as your USB drive; if not, you can manually select it. Set the *Boot selection* to the RHEL 7.9.0 ISO we just downloaded. For a modern workstation, use **GPT** as the partition scheme and **UEFI** as the target system. GPT is preferred because current Windows installations and enterprise workstations normally boot in UEFI mode, and using

the same boot mode for both Windows and RHEL avoids mixed-mode bootloader issues. Use **MBR** only if the machine is confirmed to be using legacy BIOS boot. On Windows, this can be checked from **System Information** by looking at **BIOS Mode**: choose UEFI if it says *UEFI*, and choose legacy/MBR only if it says *Legacy*.

For the file system, keep the Rufus default unless there is a specific reason to change it. **FAT32** is the safest choice for UEFI boot media because it is directly supported by UEFI firmware; **NTFS** may be used if Rufus requires it because of ISO file-size constraints. Do not choose **EXT4** for the installation USB, since it is not reliably readable by PC firmware during boot and is meant for Linux partitions after installation. Leave the cluster size at **Default**, typically **4096 bytes**, since changing it provides no practical benefit for this installer.

Once this completes, the USB is now ready for installation. Reboot your system, and *enter the Boot Menu* by pressing the relevant key. In our case, we're working on an HP workstation, so we're pressing **F9**.

### 2.6.3 Installation Destination Stage

This is the main step for dual booting, and usually the place where most novices mess up. Most of the steps are the same as the virtualization setup. The only thing we're going to have to do differently here is selecting the disk where the base system will be installed. Click on the "**Installation Destination**" option from the image below, and select the disk we chose in Subsection 2.6.1 – *nvme1n1*

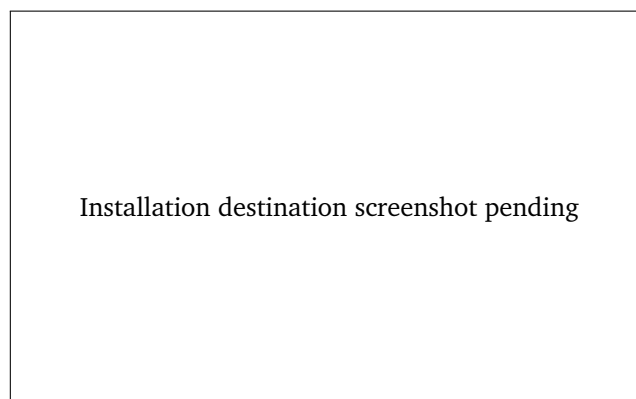


Figure 2.3: Installation Media Selection

Once the base system installs, you should remove the installation media and wait for the system to reboot.

## 2.7 Chapter Summary

This chapter established the installation baseline for deploying **RHEL 7.9.0** on a workstation intended for Cadence and other EDA workflows. We first compared the two supported deployment models: virtualization and native dual booting. Virtualization is useful for testing, training, and low-risk experimentation, while native dual booting is the preferred approach for production-style EDA work because it gives RHEL direct access to the workstation hardware, storage, memory, graphics stack, network interface, and any license-related devices.

We then identified the hardware expectations for a practical EDA workstation, including sufficient RAM, SSD/NVMe storage, CPU capacity, stable graphics support, and reliable network connectivity. The specific HP workstation used in this deployment was documented so that later installation decisions could be tied back to the actual machine configuration. The chapter also covered how to obtain the correct RHEL installation ISO, why the full DVD installer is preferred over the boot ISO, and why RHEL 7.9.0 remains suitable for toolchains that depend on older enterprise Linux compatibility.

The firmware layer was then reviewed through the distinction between legacy BIOS, UEFI, and Secure Boot. For modern dual-boot workstations, the important rule is to keep the Windows and RHEL installations in the same boot mode. In our case, the system uses **UEFI**, so the RHEL installation media and final installation should also follow the UEFI path. This avoids mixed bootloader behavior and makes the system easier to recover and maintain.

For the virtualized setup, the chapter showed how to prepare a RHEL virtual machine using Hyper-V, attach the downloaded ISO, configure the virtual firmware and DVD boot order, disable Secure Boot inside the VM, and complete the base installation. This path provides a convenient sandbox for learning the installer and validating commands before making changes to the physical machine.

For native dual booting, the chapter introduced the partitioning strategy required for a Linux workstation, including the EFI partition, root partition, and swap partition. It also explained how Windows disk numbering maps to Linux NVMe device names such as `nvme0n1` and `nvme1n1`. This mapping is critical because selecting the wrong disk during installation can overwrite the existing operating system or user data.

Finally, we created the bootable RHEL installation USB using Rufus, selected the correct ISO, and chose settings appropriate for the target workstation: **GPT** partition scheme, **UEFI** target system, and the default filesystem and cluster-size options unless Rufus requires otherwise. At the end of the chapter, the machine is ready to boot from the USB installer and proceed to the RHEL installation destination stage, where the correct target disk must be selected carefully before installation begins.





# 03

## Post Install Configurations

---

System identity, licensing, and workstation preparation

In this chapter we will discuss the post-installation configurations required for the deployed workstations. This includes setting up the appropriate hostname and username nomenclature aligned with the license provided by C2S, preparing the system identity used by the Cadence environment, and installing the relevant software packages required for the Cadence Suite.

### 3.1 Username/Hostname Configuration

After the base RHEL installation is complete, the first administrative task is to standardize the identity of each workstation. In an EDA laboratory environment, usernames and hostnames should not be chosen casually. They are used by the operating system, network services, license checks, remote administration scripts, shared storage mounts, and Cadence startup environments. If different machines follow different naming patterns, the environment becomes difficult to audit and errors in license or path configuration become much harder to diagnose.

According to the license our university has received, the workstation identity should follow the following naming convention:

`user0_X @ ECE-X.gsv.ac.in`

Here, `user0X` represents the local Linux username assigned to the workstation user, while `ECE-X.gsv.ac.in` represents the fully qualified hostname assigned to that workstation on the department network. The value of `X` should be replaced by the workstation number or seat number allocated by the laboratory administrator. For example, workstation 3 may use the username `user003` and the hostname `ECE-3.gsv.ac.in`. The important point is that the same numbering scheme must be used consistently across the user account, hostname records, license configuration, and any administrative documentation.

This convention gives every machine a predictable identity. A system administrator can immediately identify which physical workstation corresponds to which Linux account and hostname, and the Cadence license environment can be configured against a stable naming scheme. Before installing or launching Cadence tools, confirm that the selected username and hostname match the C2S license documentation and the laboratory workstation inventory. If a machine is renamed later, related configuration files, license references, and shell startup scripts may also need to be updated.

At the time of writing this document, there are officially upto 100 registered workstations for GSV students under the C2S License. The first 70 systems are reserved for the ECE department, while the remaining 30 are reserved for EE department.



# 04

## Installing and Configuring Cadence

---

System identity, licensing, and workstation preparation

- 4.1 Cadence Directory Structure**
- 4.2 Environment Variables**
- 4.3 C-Shell Configuration**
- 4.4 License Configuration**
- 4.5 Verifying Installation**
- 4.6 Launching Cadence Tools**



## **Part II**

# **RTL DESIGN FUNDAMENTALS**

---

Introduction to Verilog and basic RTL design



# 05

## Introduction to Verilog

---

System identity, licensing, and workstation preparation

### 5.1 Why Hardware Description Languages Exist

### 5.2 Verilog Program Structure

### 5.3 Modules

### 5.4 Ports

### 5.5 Nets and Registers

### 5.6 Numbers and Constants

### 5.7 Operators

### 5.8 Laboratory Exercise

Half Adder





# 06

## Combinational Logic Design

---

System identity, licensing, and workstation preparation

**6.1 Continuous Assignments**

**6.2 Dataflow Modeling**

**6.3 Conditional Operators**

**6.4 Multiplexers**

**6.5 Full Adder Design**

**6.6 Laboratory Exercise**

**4-Bit Ripple Carry Adder**



# 07

## Sequential Logic Design

---

System identity, licensing, and workstation preparation

### 7.1 Why Memory is Required

### 7.2 Clock Signals

### 7.3 D Flip-Flops

### 7.4 Registers

### 7.5 Counters

### 7.6 Laboratory Exercise

#### 4-Bit Synchronous Counter



## **Part III**

# **VERIFICATION**

---

Testbenches, simulation, waveform analysis, and coverage



# 08

## Verilog Testbenches

---

System identity, licensing, and workstation preparation

**8.1 What is Verification?**

**8.2 Testbench Architecture**

**8.3 Initial Blocks**

**8.4 Always Blocks**

**8.5 Clock Generation**

**8.6 Stimulus Generation**

**8.7 Self-Checking Testbenches**

**8.8 Laboratory Exercise**

**Counter Testbench**





# 09

## Simulation and Waveform Analysis

---

System identity, licensing, and workstation preparation

**9.1 Compilation**

**9.2 Elaboration**

**9.3 Simulation**

**9.4 Understanding Waveforms**

**9.5 Common RTL Bugs**

**9.6 Debugging Methodology**

**9.7 Laboratory Exercise**

Counter Waveform Analysis



# 10

## Coverage Analysis using IMC

---

System identity, licensing, and workstation preparation

**10.1 Why Simulation Alone is Not Enough**

**10.2 Code Coverage**

**10.3 Branch Coverage**

**10.4 Toggle Coverage**

**10.5 Functional Coverage**

**10.6 Using IMC**

**10.7 Coverage Reports**

**10.8 Coverage Closure**

**10.9 Laboratory Exercise**

**Counter Coverage Analysis**



## **Part IV**

# **LOGIC SYNTHESIS**

---

Synthesis, timing analysis, and logical equivalence checking



# 11

## Introduction to Synthesis

---

System identity, licensing, and workstation preparation

**11.1 RTL vs Hardware**

**11.2 Standard Cell Libraries**

**11.3 Liberty Files**

**11.4 Timing Constraints**

**11.5 The Synthesis Flow**

**11.6 Reading Reports**





# 12

## Logic Synthesis using Genus

---

System identity, licensing, and workstation preparation

### 12.1 Preparing RTL

### 12.2 Writing Constraints

### 12.3 Running Genus

### 12.4 Generated Outputs

- Netlist
- Timing Reports
- Area Reports
- Power Reports

### 12.5 Laboratory Exercise

Synthesizing the Counter



# 13

## Timing Analysis

---

System identity, licensing, and workstation preparation

**13.1 Propagation Delay**

**13.2 Setup Time**

**13.3 Hold Time**

**13.4 Clock Frequency**

**13.5 Critical Paths**

**13.6 Slack**

**13.7 Laboratory Exercise**

**Counter Timing Analysis**



# 14

## Logical Equivalence Checking

---

System identity, licensing, and workstation preparation

**14.1 Why Equivalence Checking Matters**

**14.2 Golden vs Revised Designs**

**14.3 Conformal Workflow**

**14.4 Laboratory Exercise**

**Verifying Counter Equivalence**



## **Part V**

# **PHYSICAL DESIGN**

---

Floorplanning, power planning, placement, clocking, routing, and verification





# 15

## Introduction to Physical Design

---

System identity, licensing, and workstation preparation

**15.1 What Happens After Synthesis?**

**15.2 Netlists and Standard Cells**

**15.3 Die, Core and IO Regions**

**15.4 Physical Design Flow Overview**



# 16

## Floorplanning

---

System identity, licensing, and workstation preparation

**16.1 Defining the Core Area**

**16.2 IO Placement**

**16.3 Macro Placement**

**16.4 Utilization**

**16.5 Laboratory Exercise**

**Counter Floorplan**



# 17

## Power Planning

---

System identity, licensing, and workstation preparation

### 17.1 Why Chips Need Power Networks

### 17.2 Power Rings

### 17.3 Power Stripes

### 17.4 IR Drop Concepts

### 17.5 Ground Networks

### 17.6 Laboratory Exercise

### Counter Power Plan



# 18

## Placement

---

System identity, licensing, and workstation preparation

### 18.1 Standard Cell Placement

### 18.2 Congestion

### 18.3 Density Optimization

### 18.4 Laboratory Exercise

### Counter Placement





# 19

## Clock Tree Synthesis

---

System identity, licensing, and workstation preparation

### 19.1 Clock Distribution

### 19.2 Clock Buffers

### 19.3 Clock Skew

### 19.4 Clock Latency

### 19.5 Laboratory Exercise

Counter CTS



# 20

## Routing

---

System identity, licensing, and workstation preparation

**20.1 Global Routing**

**20.2 Detailed Routing**

**20.3 Routing Resources**

**20.4 Signal Integrity**

**20.5 Laboratory Exercise**

**Counter Routing**



# 21

## Physical Verification

---

System identity, licensing, and workstation preparation

**21.1 Design Rule Checking (DRC)**

**21.2 Layout Versus Schematic (LVS)**

**21.3 Common Physical Violations**

**21.4 Laboratory Exercise**

**Counter Verification**



## **Part VI**

# **TAPEOUT AND BEYOND**

---

GDSII generation, packaging context, and complete flow review





# 22

## GDSII Generation

---

System identity, licensing, and workstation preparation

### 22.1 What is GDSII?

### 22.2 Manufacturing Data

### 22.3 Tapeout Process



# 23

## From Silicon to PCB

---

System identity, licensing, and workstation preparation

### 23.1 Packaging

### 23.2 Pin Assignment

### 23.3 Chip Integration

### 23.4 PCB Design Workflow

### 23.5 Why PCB Design is Different from ASIC Design



# 24

## Complete Design Flow Review

---

System identity, licensing, and workstation preparation

**24.1 Revisiting the Counter**

**24.2 RTL → Simulation → Coverage**

**24.3 Synthesis → Timing → Verification**

**24.4 Floorplanning → CTS → Routing**

**24.5 Tapeout**

**24.6 Final Flow Summary**

Verilog → Simulation → IMC → Genus → Conformal → Innovus → GDSII